

FRC Driver Lab

Product, architecture, and implementation specification

Planning version 1.0 | September 11, 2026 | Working product name

Build a desktop-browser application that lets an FRC coach run repeatable driving exercises, compare specific skills under matched conditions, and print a useful report before the next student takes the controller.

Recommended stack: TypeScript, Three.js, Rapier 3D/WASM, React, and Vite. Deliver through HTTPS on Windows and macOS. Keep simulation, scoring, and report generation on the computer. No account, application server, or database is required.

First release: four-module swerve, six-wheel differential drive, and four-wheel mecanum; USB gamepad and keyboard input; polished, scalable graphics; free drive; six core tests; session roster; PDF download and print. Drivetrain-specific extension drills follow the core release.

The hardest work is credible physics and fair measurement. A rigid-body engine supplies collision solving, but the application must implement and calibrate drivetrain forces, steering response, traction, braking, and test rules. A visually convincing scene alone does not establish training value.

This is a proposed specification, not an implemented or validated product. Numerical settings, score anchors, performance budgets, and effort estimates are initial engineering proposals. Sources were reviewed on September 11, 2026; no physical controller, competing simulator, or robot was tested for this document.

Decisions at a glance

Decision	Recommendation
Platform	Desktop web first; qualify Windows and Mac independently
Game experience	Polished, readable 3D field with force-based motion and scalable graphics
Assessment viewpoint	Fixed driver-station perspective; training can use other cameras
Main output	Skill scores, raw measurements, attempt history, and coaching priorities
Storage	Memory plus small tab-session recovery record; explicit end-session clearing
First engineering gate	Prove wired input, motion feel, frame pacing, and PDF output on actual team computers
Native fallback	Godot desktop if the prototype identifies a platform limitation native input/rendering can solve

Reading guide

Sections 1-4 cover the product, visual direction, and technology decision. Sections 5-12 specify physics, tests, and scores. Sections 13-17 cover architecture, data, performance, and verification. Sections 18-20 provide delivery priorities, risks, and sources.

1. Product purpose and scope

Users and decisions

Coach: set up one consistent assessment profile, rotate students through it, identify specific practice needs, and export results. **Student:** learn the controls, practice without risking a robot, understand a result, and repeat a focused drill. **Student developer:** maintain readable test definitions and add exercises without changing the physics engine.

The product supports driver development and provides one input into driver selection. It does not measure communication, strategy, stress tolerance at an event, or safe operation of the real robot. Coaches should observe those separately and validate promising simulator performance on the team robot.

Requirements and release boundaries

ID	Required behavior	First-release acceptance evidence
R1	Responsive 3D driving with credible inertia, traction, braking, and collisions	Physics checks and hardware performance gate pass
R2	Swerve, differential, and mecanum feel and behave differently	All three pass their motion fixtures and coach review
R3	USB controller and keyboard operation	Calibration, remapping, unplug, and focus-loss tests pass
R4	Free drive and guided practice	Unlimited practice, reset, cameras, optional visual aids
R5	Quantitative assessment	Six core tests; versioned scoring; valid, DNF, and invalid statuses
R6	Printable driver report	Local PDF download and print, with raw metrics and comparison context
R7	Temporary multi-driver session	Roster, attempt history, reload recovery, export, and clear-session action
R8	Attractive graphics that scale to available hardware	Art-quality review and frame/memory qualification pass together

Excluded initially: seasonal scoring mechanisms, robot CAD import, multiplayer, accounts, cloud rankings, robot-code execution, live robot connectivity, VR, and a course editor. These add scope without establishing assessment quality.

Proposed product success measures

- At least 8 of 10 pilot users reach free drive within three minutes after connection, without developer assistance.
- A coach can collect and export a six-test screening result in approximately 12-15 minutes per student, including setup.
- At least 90% of pilot report readers can identify the student's two next practice priorities without explanation.
- Every displayed score can be recalculated from its recorded metrics and rubric version.
- Reliability and real-robot transfer are measured in a pilot; neither is assumed from these usability targets.

2. Platform and technology decision

Options evaluated

Option	Fit for this application	Main cost or limitation	Decision
TypeScript + Three.js + Rapier web	Strong fit for modest 3D scenes, custom tests, accessible forms, and local reports	Custom wheel dynamics; browser input and frame pacing need qualification	Recommended
Godot native	Strong scene tooling, native deployment, and input integration	Separate OS builds; installer approval; report layout/export needs additional work	Best fallback
Godot web	Reuses game-engine scenes in browser	WebAssembly/WebGL 2 and web export constraints; browser gamepad mapping still applies	Viable if team already uses Godot
Unity native/web	Mature game workflow, especially with an experienced Unity team	Larger tooling and distribution footprint; commercial terms need project review	Choose if existing expertise changes delivery cost
Separate Swift/macOS and Windows apps	Good OS integration	Duplicate rendering, physics, controls, and QA effort	Poor fit

These are engineering judgments for this scope, not measured engine benchmarks. Three.js provides a WebGL 2 renderer; Rapier exposes a JavaScript/WASM physics engine. Godot's web documentation describes browser/export constraints, and Unity's web input remains subject to the browser Gamepad API. [Three.js renderer](#), [Rapier setup](#), [Godot web export](#), [Unity web input](#).

Why web first

One application can run on team Windows laptops and Macs, without maintaining two native interfaces. Standard HTML supports clear forms, responsive setup pages, accessible result tables, and print layouts. A detailed single-robot arena with restrained background complexity is an appropriate starting workload for this approach, subject to profiling. Web delivery does not require a crude visual style: the art direction, materials, lighting, and optimization are explicit deliverables.

Use React only for menus, session state, and reports. Keep the real-time simulation loop outside React rendering. Start with a direct Three.js scene and the WebGL 2 renderer; make WebGPU optional future work. Freeze exact dependency versions at the prototype gate.

Go/no-go rule

Before building the full test battery, prove USB input on the actual controller models and OS versions, all three drivetrain prototypes, stable 60 fps at 1080p on the agreed baseline, and offline/local report export. If a failure is browser-specific, investigate Godot native. If it is the wheel model or course design, changing engines will not fix it automatically. An Electron wrapper also does not remove the need to qualify OS controller support.

3. Experience, cameras, and free drive

Session workflow

1. **System check:** graphics support, controller response, display size, and short motion benchmark.
2. **Coach setup:** session label, driver aliases, robot preset, control scheme, camera, difficulty, and suite.
3. **Calibration:** identify controller, verify axes/buttons, set deadzones, and show a live robot-direction preview.
4. **Familiarization:** three minutes of free drive; extend if needed and record duration.
5. **Practice or assessment:** instructions, demonstration, countdown, attempt, and concise result.
6. **Review:** raw measurements, skill scores, path replay, and the next suggested drill.
7. **Export and rotate:** individual or batch reports; switch driver without changing the locked profile.

Provide large **Free Drive**, **Practice a Skill**, and **Run Assessment** choices. Keep connection state, selected robot, control frame, and pause available throughout. A coach override is a session setting, not an authentication or anti-cheat system.

Cameras and visual judgment

The default assessment uses a fixed driver-station camera behind the near wall, looking down the field. Lock position, target, field of view, viewport aspect, and zoom in the assessment profile. Display a clear robot front marker and bumpers. Use stable field markings, shadows, perspective, and occlusion; avoid motion blur, camera shake, and exaggerated visual effects.

This measures **screen-based distance and alignment judgment**, not clinical depth perception or stereoscopic vision. Screen size and viewing distance affect the task. Standardize the physical station for selection sessions, record the display dimensions/resolution, and allow a separately labeled accommodation profile.

Free drive offers driver-station, orbit, chase, and overhead views. Assessment disables overhead maps, target-distance readouts, ghost paths, and camera changes. Replays may show an overhead path after an attempt.

Free-drive content

Use a generic training arena with an open lane, slalom markers, docking outlines, narrow gates, and a turning area. Offer speed limiting, a controller overlay, optional trajectory trail, heading indicator, quick reset, and braking/coasting demonstrations. No timers or scores are required. Let students compare robot-relative and field-relative swerve, then explain why a given stick motion produces different behavior.

Responsive interface

Design gameplay for a landscape laptop/desktop viewport, initially at least 1280 x 720. Smaller screens can view instructions and reports but should not receive qualified assessment status until tested. Letterbox the assessment viewport rather than changing its field of view. Keyboard-accessible menus, text plus icons, high contrast, and remapping should be standard.

Support adjustable deadzones, reduced effects, scalable text, and selectable report text. Mirror sound cues visually. Accommodations affecting the driving task use a named comparison profile printed in the report. An equivalent nonvisual driving assessment would require a separately designed protocol.

3A. Visual direction and graphics requirements

Look and feel

Aim for a polished robotics training game in a clean competition-practice venue. Use believable proportions, physically based materials, soft directional lighting, grounded contact shadows, and a coherent interface. Graphics quality is a release criterion alongside responsiveness and memory use. Shipping development cubes and a flat-colored floor does not satisfy the brief.

Robot: recognizable swerve modules, differential wheels, and mecanum rollers; beveled chassis edges; bumper fabric and seams; restrained metal roughness; readable front markings. Animate wheel rotation and module steering from actual physics state. Use baked normal detail for small fasteners instead of unnecessary geometry. Keep collision dimensions aligned with visible bumpers.

Environment: textured carpet at correct physical scale, crisp field tape, believable gate materials, gently varied surfaces, and a finished practice-venue backdrop. Concentrate detail near the robot and driving surface. Keep the background visually quiet. Avoid glossy carpet, noisy textures, excessive bloom, depth-of-field blur, or decorative clutter that hides test targets.

Interface: consistent typography, spacing, icon family, restrained color palette, attractive robot-selection cards, clear controller status, and a compact in-game HUD. Use deliberate transitions in setup/results. Gameplay overlays must leave the driving sightline clear. Reports should follow the same visual identity while remaining legible in grayscale.

Asset and rendering pipeline

Author original or properly licensed assets in Blender and export glTF/GLB. Build reusable materials, geometry detail levels, texture atlases where useful, and compressed textures after compatibility testing. Use physically based metallic/roughness materials, an environment light/reflection map, a shadowed key light, and baked ambient detail. Tune tone mapping and exposure as a complete scene. Preload/decode assets and warm shaders before trials. Three.js supports the proposed material workflow and glTF loading. [Three.js materials](#), [glTF loader](#).

Keep render models separate from simplified collision geometry. High-quality render details must not increase physics cost. Instance repeated markers and field fixtures. Inspect representative near, far, and driver-station views on the weakest qualified machine as well as a higher-end computer.

Scale beauty to the hardware

Tier	Proposed treatment
Performance	Same art, silhouettes, markings, materials, and required shadow cues; lower texture resolution, mesh detail, and decorative lighting cost
Standard	Sharper textures, improved shadow filtering, fuller background detail, and higher render resolution within budget
High	Highest useful asset detail and shadow quality; optional subtle ambient occlusion and decorative effects if measured headroom allows

Choose the highest tier that passes a representative preflight with headroom. Dynamic resolution may operate in free drive; freeze resolution, tier, exposure, and camera during scored trials. If performance fails, stop/requalify instead of changing visual cues mid-test. Include graphics tier and render resolution in comparison metadata; initially require matched tiers for selection comparisons until equivalence is validated.

Visual acceptance: no obvious placeholder assets, stretched textures, flickering tape, floating robots, unreadable bumpers, or distracting shadow artifacts. Coach and students review a finished scene and setup/results screens on both target platforms. Art review and performance review must pass together; neither excuses failure of the other.

4. Controller and keyboard specification

Input compatibility

Use `navigator.getGamepads()` through the Gamepad API, served over HTTPS. Prompt the user to focus the page and press a controller button; discovery may require interaction. Prefer standard mappings but verify each axis and button. Do not use WebUSB as the default controller interface. [MDN Gamepad guidance](#), [Mozilla secure-context guidance](#).

A USB Xbox controller on Mac is a qualification item, not a universal promise. Apple documents compatible wired controllers generally, while its Xbox guide lists particular wireless models and Bluetooth pairing. Test the exact model, firmware, data cable/adaptor, OS, and browser. The browser may not expose whether transport is USB or Bluetooth; record the coach-confirmed connection type. [Apple wired controller guidance](#), [Apple Xbox guidance](#).

Proposed default bindings

Action	Gamepad	Keyboard
Holonomic translation	Left stick	W/S forward/back, A/D left/right
Holonomic rotation	Right stick horizontal	Q/E left/right rotation
Differential arcade	Left stick vertical + right stick horizontal	W/S + A/D steering
Differential tank	Left/right stick vertical	W/S left side, Up/Down right side
Precision mode	Left bumper, hold	Shift, hold
Brake / swerve X-lock	Right bumper, hold	Space, hold
Pause	Menu	Escape
Reset	UI action after pause	UI action after pause

Field-relative control is available for swerve and mecanum. Tank and arcade are control schemes for differential drive, not separate drivetrains. Zero heading and control-frame switching are permitted in practice; locked assessment settings prevent accidental changes.

Calibration and failure handling

Capture neutral drift, full axis travel, axis inversion, and intended bindings. Start with a radial stick deadzone of 0.08 and an adjustable response exponent of 1.5; these are proposed defaults. Rescale remaining travel to full output. Normalize diagonal translation. Keyboard commands use configurable acceleration/deceleration ramps and remain a distinct comparison group.

Choose one active input device before a trial. Poll every animation frame, time-stamp the sample, and consume it at physics ticks. On disconnection, loss of focus, hidden tab, or graphics-context loss: zero input, pause, and invalidate the assessment attempt. Reconnection requires neutral controls and an explicit restart. In free drive, allow resume. Do not interpret an unchanged gamepad timestamp alone as a disconnected device.

5. Physics model and realism boundary

Minimum credible simulation

Use a 3D dynamic chassis with mass, center of mass, inertia, bumper collision geometry, and wheel contact points. Apply forces and torques at the wheel locations. The physics solver integrates movement and resolves impacts; input must never directly teleport or set the chassis pose during a running trial.

For the first release, use flat-floor wheel contact with bounded load assumptions. Suspension travel, ramps, tipping fidelity, battery sag, and structural damage are later extensions. The chassis still collides in 3D, but the product must not claim validated ramp or tip behavior. Reset and start-state initialization are the only permitted pose teleports.

Proposed force pipeline

1. Shape input and convert the selected control frame to robot-relative commanded chassis speed.
2. Convert that command into per-wheel speed and steering targets, respecting drivetrain limits.
3. Evolve steering and drive response with finite rates and acceleration/current-inspired force limits.
4. Compute contact-point velocity from chassis translation and angular velocity.
5. Resolve velocity into rolling and lateral directions; derive longitudinal drive/brake force and lateral scrub resistance.
6. Limit the combined force by traction and available wheel load; apply forces at contact points.
7. Step the rigid-body world and collect collision and test events.

An initial wheel controller may use $F_{drive} = \text{clamp}(k_v * (v_{target} - v_{contact}), -F_{max}, F_{max})$, with explicit wheel-speed and force envelopes. This is an effective training model, not a full motor electrical simulation. For anisotropic traction use $(F_{long} / (\mu_{Long} * N))^2 + (F_{lat} / (\mu_{Lat} * N))^2 \leq 1$. Handle near-zero wheel load without division; scale force to the ellipse boundary when exceeded.

Contact implementation choice

Avoid counting floor friction twice. If custom wheel forces represent horizontal traction, disable chassis-floor horizontal friction while retaining appropriate normal support and bumper-wall contacts. Calibrate the support and force application together so the model does not create artificial pitching or lateral sticking. Exclude floor/support contacts from driver collision penalties.

Use swept trigger tests and continuous collision detection where appropriate for fast robot/obstacle contacts. Do not trust render meshes as precise colliders. Keep bumper extents visible and consistent with physics extents.

Calibration evidence

Measure a real team's maximum speed, acceleration, neutral coast, commanded braking distance, turn response, and lateral/diagonal response where supported. Fit coefficients to several runs and validate on held-out runs. Until then, label each configuration **generic, uncalibrated preset**. Never describe all swerve robots as having one universal feel.

6. Drivetrain behavior and configuration

Required drivetrain distinctions

Drivetrain	Motion and model requirements	Key validation case
Four-module swerve	Independent steering and drive; finite steering rate; wheel-speed saturation; robot/field-relative modes	Translate while rotating; reverse direction without instantaneous module steering
Six-wheel differential	Fixed wheel axes; left/right wheel targets; lateral scrub; realistic turning resistance	No commanded strafe; straight, arc, pivot, coast, and reverse
Four-wheel mecanum	Fixed wheel axes with roller-direction force mapping; asymmetric traction and lateral efficiency	Strafe and diagonals without behaving like steering modules

Later presets may add four-wheel differential, traction/omni mixes, H-drive, and other mechanisms if teams need them. Do not include exotic configurations simply to increase the menu count.

Coordinates and kinematics

Use an internal robotics frame: +X forward, +Y left, +Z up, positive yaw counterclockwise. Centralize conversion to Three.js coordinates: render (X, Z, -Y). Test every basis vector and heading sign. Field coordinates do not change when the driver changes sides; a separate driver-perspective transform defines controls.

For swerve module position (x_i, y_i) , its desired planar velocity is $(v_x - \omega y_i, v_y + \omega x_i)$. Compute steering angle and wheel speed, scale all targets together if any exceeds its maximum, and optimize reversal when that reduces steering travel. Keep a small-speed angle-hold threshold to prevent neutral jitter. Actual steering follows the rate limit. Field-relative inputs rotate by negative robot heading before this calculation.

For differential drive with effective track width b , $v_{\text{Left}} = v_x - \omega b/2$, $v_{\text{Right}} = v_x + \omega b/2$; commanded lateral speed is zero. Tune effective scrub and support for six-wheel/drop-center behavior. For mecanum, specify roller orientation and wheel order explicitly and test the resulting sign matrix. WPILib provides useful reference conventions and kinematics; these equations do not by themselves model traction or dynamics. [WPILib kinematics](#), [swerve reference](#).

Robot preset contract

Store dimensions including bumpers, mass, center of mass, inertia policy, wheel locations/radii, maximum speed, drive-force envelope, steering-rate limit, longitudinal/lateral friction, neutral behavior, and calibration provenance. Version the complete preset.

A starting development fixture may use a 0.90 m square bumper footprint, 55 kg mass, 0.60 m wheelbase/track, and 4.5 m/s swerve maximum speed. These values are illustrative development choices, not FIRST size/weight rules or measured team specifications. Publish separate values for the other drivetrains after the prototype; do not invent calibrated equivalence.

7. Assessment protocol and test geometry

Two assessment lengths

Screening: one recorded trial of each core test after familiarization; approximately 8-10 minutes of exercises and transitions, or 12-15 minutes including setup. Report one-trial results as preliminary and do not issue a consistency score.

Full assessment: three recorded trials per core test, with a common practice exposure and planned breaks. Budget 25-35 minutes per student for the initial design. Validate these timings in the pilot. Rotate test order using a fixed published schedule to reduce fatigue/order effects; preserve the schedule identifier in the report.

For each test, allow one unscored demonstration/practice attempt before scored attempts. Use equal practice exposure for comparative selection. Retain all scored attempts, including DNF. Permit one replacement per scheduled trial only for a recorded technical invalidation; repeated technical failures stop qualified assessment until resolved. Student abort/reset is DNF, not a free retry.

Shared geometry

Use a generic 16 m x 8 m floor with perimeter walls; this is a training arena, not a regulation claim. Let bumper width be B and bumper length L . Configure gates and bays relative to those dimensions. Rebuild geometry separately for each test. Verify that all required paths are physically feasible for the selected drivetrain and preset before issuing its qualified suite.

Every definition includes start pose, legal corridor, ordered checkpoints, target poses, dwell conditions, time limit, seed, assistance policy, metric extraction rules, and score profile. Freeze the published set of target positions and headings within a comparison session. Randomization may vary practice; assessment variants need independent calibration before being called equivalent.

Core and extension tests

Test	Main skill	Availability
T01 Distance docking	Screen-based distance judgment and final alignment	All
T02 Orientation shuttle	Control when facing away, toward, or sideways	All; control-frame-specific
T03 Speed slalom	Fast movement with directional control	All; drivetrain-specific course profile
T04 Precision gates	Bumper awareness and precise maneuvering	All
T05 Sprint and stop	Speed regulation and braking judgment	All
T06 Cue and recover	Response to an unexpected direction cue	All; drivetrain-specific feasible exits
T07 Translate and face	Independent translation and heading control	Swerve/mecanum extension
T08 Repeated cycles	Sustainable pace and consistency	Optional extension

8. Test definitions: distance, orientation, and speed

T01 - Distance docking

Procedure: three docking bays at near, middle, and far depths from the locked camera. Start from a marked pose for each segment; drive the bumper footprint into a bay of size $(B + 0.30 \text{ m}) \times (L + 0.30 \text{ m})$. Target headings and approach lanes are stored in the test file. Limit the full trial to 60 seconds.

Completion: each bay requires the complete footprint inside, center error at most 0.12 m, heading error at most 10 degrees, speed below 0.15 m/s, and continuous dwell of 0.75 seconds. The next target activates only after a valid dwell. These initial tolerances need pilot tuning.

Metrics: time from trial start through final dwell; mean center error over the final 0.25 seconds of each valid dwell, then averaged across bays; corresponding absolute heading error; collisions. Keep near/middle/far errors separately for coaching. No distance readout, aiming line, or overhead view during assessment. This includes maneuvering skill and is not a pure visual-acuity test.

T02 - Orientation shuttle

Procedure: four short legs begin with the robot facing 0, 90, 180, and 270 degrees relative to the field, in a fixed balanced sequence. Each leg uses an ordered destination gate and a dwell endpoint. Orientations are preset between segments while input is disabled; this reset is part of the test. Limit the trial to 60 seconds.

Completion: reach each endpoint in order at below 0.20 m/s for 0.50 seconds. Gates have adequate approach space for differential drive. Field-relative holonomic and robot-relative control receive distinct profile IDs; neither is presented as intrinsically better.

Metrics: elapsed time, path error, contacts, and wrong-direction duration. For the first second of each leg, accumulate time when speed exceeds 0.10 m/s and velocity has a negative projection onto that leg's initial intended travel direction. Report 180-degree-leg delay separately. Use fixed route polylines appropriate to the drivetrain; penalize neither necessary differential turns nor purposeful heading changes as wrong-way translation.

T03 - Speed slalom

Procedure: travel down and back through alternating ordered gates. Initial fixture: five gates, approximately 2 m spacing, with enough lateral clearance for the tested robot. Swerve/mecanum and differential use separately calibrated routes. Limit the trial to 45 seconds.

Completion: pass every gate in order and direction, then settle in the finish box. Skipping a gate never advances progress. Crossing between markers outside the legal opening is not a pass.

Metrics: elapsed time, progress-indexed cross-track RMS error, contacts, corridor departures, peak speed, and route length. Peak speed is diagnostic only; it does not earn points by itself. A high-speed crash must not outperform controlled completion.

9. Test definitions: precision, braking, and recovery

T04 - Precision gates

Procedure: negotiate three passages: straight, offset, and angled docking. Start at a fixed pose. Initial gate clearance is $B + 0.20$ m; provide a turning pocket large enough for differential drive. Limit the trial to 60 seconds.

Completion: full bumper footprint passes ordered gate planes inside their openings; settle in the final bay at less than 0.15 m/s for 0.75 seconds. Check swept geometry so a fast crossing cannot skip a trigger or pass through a post.

Metrics: elapsed time; RMS lateral offset at the gate crossing planes relative to each opening's center; final absolute heading error; collisions and corridor departures. Show nearest bumper clearance as feedback, not an extra score that rewards hugging obstacles.

T05 - Sprint and stop

Procedure: accelerate along a marked straight lane, cross an entry line at a prescribed speed band, and stop at a visible target. Run three segments with published target distances. Initial band: 65-80% of the preset maximum speed. Ensure available run-up and stopping room exceed the model's validated requirements. Limit the trial to 45 seconds.

Completion: each segment must cross its entry line in the speed band and settle below 0.10 m/s for 0.75 seconds. Measure stop position at the first qualifying dwell. A segment outside the band is an unmet condition; it cannot earn an easy score by crawling. Failure to complete all three segments within the limit is DNF.

Metrics: mean absolute signed stopping error relative to the target, mean overshoot beyond the target, elapsed time, entry speeds, and contacts. Define longitudinal position using the front bumper plane projected onto the lane direction. Record early stops as negative signed error. Brake/precision mode availability is locked and printed in the profile.

T06 - Cue and recover

Procedure: drive through an approach gate, then receive a visible left/right destination cue after a seeded delay. Use a three-event balanced sequence. Targets and route openings remain feasible for the selected drivetrain. Limit the trial to 60 seconds. No sudden opponent collision is needed to test recovery in the first release.

Completion: take the indicated branch, cross its ordered gate, and settle at the endpoint; wrong branches must be corrected within the limit. Begin reaction timing only after the cue has been presented in a rendered frame. Record both presentation and subsequent input timestamps with the same monotonic clock.

Metrics: median cue-to-correct-input latency; count of wrong branches; elapsed time; collisions. Correct input is a cue-consistent command above 0.20 magnitude sustained for 0.15 seconds, after the cue, according to the profile's control frame and initial heading. Pre-cue held input is not a response. Report display/input latency context; do not describe this as a medical reaction-time assessment.

10. Optional drills and coach use

T07 - Translate and face

For swerve and mecanum, follow a square or circular translation path while keeping the front marker aimed at a specified target. Limit the trial to 45 seconds. Record heading RMS error, path RMS error, completion time, and contacts. Use explicit heading-controller assistance settings. Score this as an extension; it must not reduce a differential driver's core score for lacking strafe capability.

T08 - Repeated cycles

Repeat a route between two docking bays for 90 seconds. Count only complete, legal cycles. Show completed cycles, median cycle time, cycle-time spread, docking error, and contacts. Do not attach a mechanism or seasonal scoring dependency. Treat this as sustained practice until its own rubric has been calibrated.

Recommended additions, in priority order

Priority	Feature	Coach/student benefit
1	Post-attempt path replay with control overlay	Makes overshoot, reversals, and unnecessary corrections visible
1	Targeted drill recommendation	Turns a low score into a specific next action
1	Coach notes and teammate feedback	Captures observations the simulator cannot measure
2	Team robot calibration profile	Helps practice resemble the robot students will drive
2	Matched session comparison and batch PDF	Reduces administration during tryouts
2	Practice ghost from the student's earlier run	Shows improvement without a public ranking
3	Driver-perspective changes and partial occlusion	Adds event-like challenge after core skill reliability is established
3	Coach callouts / driver-operator coordination	Adds communication practice; needs separate observation rubric
3	Moving obstacles and light contact recovery	Adds traffic awareness once collision behavior is validated

Coach operating guidance

Run a novice through free drive before recording a score. Keep the same controller, screen, camera, robot preset, and assistance policy across a selection group. Allow breaks. Separate a student's current proficiency from improvement after instruction. Do not let a single overall number override reliable control, communication, judgment, or observed real-robot performance.

The report should recommend one concrete drill per weak area. Examples: large far-bay error -> repeat far docking at limited speed; wrong-way motion at 180 degrees -> robot-relative return shuttles; high stopping overshoot -> entry-speed-controlled stopping; large between-trial variation -> repeat an easier course before increasing speed.

Existing products and differentiation

MoSim advertises FRC robots, scores, modding, and a season-based environment. xRC publishes seasonal releases and multiplayer server listings. DSIM describes a browser-based 2D FTC simulator. These are useful references for practice and onboarding, but this review did not verify their assessment/report features hands-on. The proposed differentiator is a transparent, season-independent assessment protocol with reproducible reports, not a claim that no competitor has scoring. [MoSim](#), [xRC](#), [xRC servers](#), [DSIM](#).

11. Scoring model: transparent and versioned

Normalize individual measurements

For a lower-is-better metric x , define a good anchor G and weak anchor W , where $W > G$:

$$N(x; G, W) = 100 * \text{clamp}((W - x) / (W - G), 0, 1)$$

This is a proposed criterion-referenced scale, not a percentile or validated population norm. Thresholds belong to the test/profile version and are frozen before an assessment. Never infer them from the current group, which would change a student's score depending on who else participates.

Per-trial scoring and contacts

For a completed trial, calculate $\text{base} = \sum(\text{weight}_i * N_i)$ with weights summing to 1. Then $\text{score} = \text{clamp}(\text{base} - P, 0, 100)$ where $P = \min(40, 4 * C_{\text{minor}} + 12 * C_{\text{major}} + 8 * C_{\text{boundary}})$.

A collision episode begins on bumper contact with a scored obstacle, excludes floor/support contacts, and ends after 0.30 seconds of separation. Use its maximum pre-impact normal closing speed: below 1.0 m/s is minor, at least 1.0 m/s is major. Sustained pushing remains one episode and consumes elapsed time. Initial overlaps are setup faults, not driver penalties. An illegal-corridor departure starts when any bumper footprint lies outside the allowed polygon; it ends after the footprint remains inside for 0.30 seconds. Exiting the arena or overturning the robot ends the trial as DNF.

Avoid duplicate penalties: classify a wall contact as a contact event; do not also issue a boundary penalty for solver penetration at the same physical wall. Boundary events refer to marked legal-corridor limits. Contact thresholds and penalty values are provisional and require pilot review.

Trial outcomes

Status	Treatment
Completed	Apply metric weights and penalties; retain raw values
DNF	Score 0; retain elapsed time, progress, and cause; included in aggregation
Invalid: technical	No score; excluded with visible reason and permitted replacement
Not attempted	Missing; prevents a complete core result
Practice	Feedback only; excluded from assessment

Timeout, student reset, student abort, and unmet required conditions are DNF. Focus loss, device disconnection, invalid starting geometry, or a performance interruption are technical invalidations. Do not silently drop poor trials or award partial-completion bonuses. A full assessment requires three scored outcomes per core test, where completed and DNF both count.

12. Initial rubrics, aggregation, and interpretation

Proposed metric weights and anchor pairs

Each pair below is G / W; lower values are better. These are development fixtures for the initial arena, not final coaching standards. Calibrate separately by drivetrain, control frame, course, input class, and assistance profile.

Test	Weighted metrics with initial G / W
T01	45% center error 0.02 / 0.12 m; 25% heading 1 / 10 deg; 30% time 20 / 60 s
T02	40% wrong-way time 0 / 2 s; 30% path RMS 0.10 / 0.80 m; 30% time 20 / 60 s
T03	45% time 15 / 45 s; 55% path RMS 0.10 / 0.80 m
T04	50% gate offset RMS 0.01 / 0.10 m; 25% heading 1 / 10 deg; 25% time 20 / 60 s
T05	55% stop error 0.05 / 0.75 m; 25% overshoot 0 / 0.50 m; 20% time 15 / 45 s
T06	40% cue latency 0.30 / 1.50 s; 35% wrong branches 0 / 3; 25% time 20 / 60 s

For T03/T02, path RMS is measured against the intended segment selected by ordered progress, not the nearest part of the entire route. Sample at fixed traveled-distance intervals of 0.05 m so waiting does not dilute path error; elapsed time handles pauses. Retain stationary-start behavior separately. Never normalize a missing metric into a perfect score.

Driver-level results

For the full assessment, each core skill score is the median of its three trial scores. Also show all attempts, min/max, contacts, and completion count. The optional **Core index** is the equally weighted mean of the six skill scores, only when all six are present. Screening displays each single-trial score as preliminary and omits the Core index. Do not rank across incompatible profiles or renormalize away a missing test.

Consistency is separate: $\text{Consistency} = \text{clamp}(100 - 2 * \text{mean}(\text{test max score} - \text{test min score}), 0, 100)$, using three trials for every core test. Display it only for the full suite and label it **repeatability**, not proficiency. Consistently low performance can produce high repeatability. Do not add it into the Core index.

Worked example

T01 has center error 0.07 m, heading error 4 degrees, time 36 s, and one minor contact. Normalized values are 50, 66.67, and 60. The score is $0.45 * 50 + 0.25 * 66.67 + 0.30 * 60 - 4 = 53.17$, displayed as **53.2**. Compute with full precision; round only for presentation.

If six skill medians are 72, 65, 81, 68, 74, and 60, the Core index is **70.0**. These are illustrative values. Use descriptive raw results and coaching advice; defer letter grades, pass/fail cutoffs, and percentile claims until the pilot supports them.

13. Driver report card and temporary sessions

Report contents

Generate a one-page summary plus optional detail pages. Include driver alias/name as entered, session/date, assessment type, qualified/provisional status, profile ID/hash, robot and physics versions, control mode, device and transport declaration, camera/display context, assistance settings, and scoring version.

Show the six scores in a labeled table with horizontal bars, all trial scores, raw metric units, completion counts, and contact counts. Show the optional Core index and repeatability only when eligible. Include strengths, two suggested drills, coach notes, technical invalidations, and an explicit **incomplete** label when required evidence is missing. A report must not turn missing data into zero without the DNF rule.

Illustrative summary layout

Driver: Practice A	Full assessment	Illustrative data only
Skill	Score / 100	Coaching focus
Distance docking	72	Compare near and far stopping error
Orientation control	65	Repeat return shuttles
Speed control	81	Maintain pace while avoiding contacts
Precision maneuvering	68	Reduce corrections at offset gates
Braking judgment	74	Practice the same entry-speed band
Cue and recovery	60	Repeat cue drill at lower practice speed
Core index	70.0	Matched-profile comparison only

The example above shows summary values, not a complete synthetic assessment record. Production reports also print the attempts and provenance needed to explain each score. The report should state that the simulator is one part of a coach's evaluation.

Export implementation

Create a pure `ReportModel` from frozen results. Render an HTML preview and generate PDF locally with `pdf-lib` and an embedded font; no remote conversion service. The library supports browser-side PDF creation. Use explicit page layout and wrapping, including long names and notes. [pdf-lib](#).

Provide separate **Download PDF** and **Print** buttons. Print opens a report-only layout using browser print facilities; the user selects a printer or Save as PDF. Support US Letter and A4, grayscale, selectable text, page numbers, and a batch export with one driver per summary page. Include the profile version on every detail page.

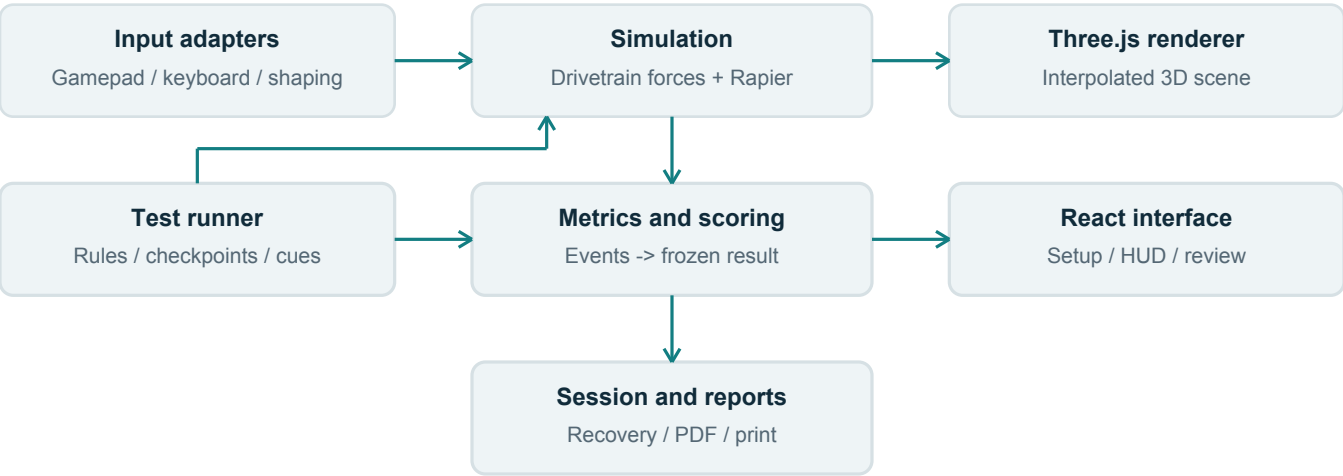
Storage lifecycle

Keep active telemetry/replay in memory. Save compact roster, completed metrics, results, settings, and notes to `sessionStorage` after state changes; exclude full replay streams. This supports reload recovery within a tab. Browser restoration may restore tab storage, so enforce an application expiry of eight hours since session creation and provide **End session and clear**. [MDN sessionStorage](#).

Handle storage failures without losing in-memory results: display a recovery-unavailable notice and preserve export. Optional explicit JSON export/import supports later comparison without a database. Exported files persist until the user deletes them. No analytics or student-data upload is required; static hosting may retain ordinary request logs.

14. Application architecture

Runtime data flow



Module responsibilities

Module	Contract and ownership
Input	Raw devices -> normalized time-stamped commands; no scoring logic
Simulation	Owns fixed clock, world, drivetrains, contacts, and snapshots
Test runner	State machine, cue schedule, ordered gates, dwell, timeout, validity
Telemetry	Tick/event logging, replay samples, metric extraction
Scoring	Pure metrics + rubric -> scores, explanations, and eligibility
Session	Drivers, locked profile, attempts, recovery, export lifecycle
Presentation	Three.js scene, React menus/HUD, report layouts

Keep course definitions, robot presets, and rubrics as versioned data with schema validation. Unknown schema versions fail with an actionable message. The renderer reads state; it never owns collisions or decides whether a target was reached. Reports read frozen results; they never rerun physics.

State and timing

Use setup -> calibrated -> ready -> countdown -> running -> completed/DNF/invalid -> review. Only running advances assessment time. A false start cannot move the chassis during countdown. Reset restores all drivetrain internal state, velocities, contacts, cues, counters, and random generator state.

Run physics at a fixed 120 Hz initially, interpolate rendering, and advance scoring from simulation ticks. Limit catch-up to eight ticks per animation frame. If the backlog cannot be caught up or a frame stalls for over 250 ms, invalidate the assessment and record a performance interruption. Never silently slow the simulation to preserve a favorable score. T06 also records wall-clock cue/input timestamps for reaction metrics.

Start single-threaded with profiling. Move physics to a Worker only if measurements justify synchronization complexity. Asset loading, PDF construction, and expensive React work occur outside running trials.

15. Data contracts, files, and reproducibility

Proposed source structure

```
src/
  app/          setup, roster, assessment, results
  input/        keyboard, gamepad, calibration, shaping
  sim/          loop, world, contacts, coordinates
  drivetrains/  swerve, differential, mecanum
  tests/        runner, geometry, cue scheduling
  metrics/      reducers and measurement definitions
  scoring/      normalization, penalties, aggregation
  reports/      model, HTML print, PDF
  session/      memory store, recovery, import/export
content/
  robots/      versioned physical presets
  courses/     geometry, checkpoints, legal corridors
  rubrics/     weights, anchors, compatibility keys
tests/         unit, physics, scenario, browser fixtures
```

Core data contracts

Record	Required fields
Session	schemaVersion, id, createdAt, expiresAt, drivers, profile, results
Profile	robotPreset/hash, physicsVersion, courseVersion, rubricVersion, inputClass, controlFrame, binding/shaping hash, assistance, camera, graphicsTier, renderResolution, difficulty, seedSet
Attempt	id, driverId, testId, scheduledTrial, status, cause, startedAt, durationTicks, wallDuration, metrics, events, performance, score breakdown
Input sample	monotonic timestamp, device slot, axes/buttons, normalized command, consumed tick
Telemetry sample	tick, pose, velocities, wheel/module states, input, active checkpoint
Event	tick, type, source IDs, position, relevant value, validity information
Report model	schemaVersion, frozen identity/context, skill results, eligibility, notes, export timestamp

Reject nonfinite numeric values, malformed imported JSON, out-of-range physics values, impossible geometry, unknown enum values, and duplicate IDs. Treat imported names/notes as text, never HTML. Cap import size, driver count, and note length. Report notes can wrap across additional pages.

Recording budget

Keep exact command/tick records and events for the current attempt; sample pose replay at 30 Hz. Set a provisional 25 MB session replay budget and discard the oldest replay only after informing the user that scores and metrics remain. Compact result recovery should stay below 2 MB. These are application budgets, not promised browser storage quotas.

Reproducibility contract

Persist software build ID, dependency/physics version, all preset data hashes, seed, and ordered events. Rapier documents deterministic WASM behavior under identical conditions, but custom JavaScript math and initialization order can break application-level determinism. Therefore require same-build score repeatability and tolerance-based motion checks across platforms; do not promise bit-identical whole-application replay. Use stored poses for review playback. [Rapier determinism](#).

A local profile hash detects accidental configuration differences; it is not a tamper-proof certification. This product has no trusted remote authority or global competitive leaderboard.

16. Performance and deployment

Proposed performance qualification

Agree on one actual Windows integrated-GPU laptop and one actual Mac before the prototype. Record CPU/GPU, OS, RAM, power mode, browser, controller, display, and adapters. A proposed development baseline is an Apple M1-class Mac and a recent Windows laptop with integrated graphics and 8 GB RAM; minimum supported machines remain unverified until tested.

Measure	Initial target and measurement
Rendering	Sustained 60 fps at 1920 x 1080 output; highest qualified graphics tier and recorded internal resolution
Frame pacing	p95 frame interval ≤ 20 ms; no >250 ms trial stalls
Physics	p95 two-tick work ≤ 4 ms at 60 Hz rendering
Input response	Median physical input-to-visible motion <80 ms, p95 <120 ms on qualified USB setup
Memory	Target <400 MB CPU-side application footprint; separate ≤ 128 MB estimated GPU texture budget on baseline
Session stability	60-minute, 20-driver soak without crash or lost completed results
Initial transfer	Target ≤ 30 MB compressed base app + training scene; load optional high-detail assets before play

Internal input timestamps do not measure USB-to-display latency. Use high-frame-rate video of physical input and screen movement, or a suitable hardware measurement setup. Measure p95 frame timing over representative courses after shader warm-up, with both cold startup and sustained load tested separately.

Rendering and qualification rules

Optimize a finished-looking scene: use mesh detail levels, instanced markers, compressed textures, capped pixel ratio, baked ambient detail, and carefully budgeted real-time shadows. Initial profiling budgets are 200 draw calls and 500,000 visible triangles on Standard; treat these as tuning guides, not guarantees or substitutes for measured frame time. Estimate GPU texture cost including mipmaps; monitor resource counts and OS/process memory because browser memory APIs do not consistently expose total GPU use. Release unused meshes, textures, and render targets when switching scenes.

Compile shaders and load assets before countdown. Select the highest visual tier that passes preflight with approximately 20% rendering-cost headroom; freeze quality during assessment. Persistent p95 failure marks the station unqualified and routes to practice until corrected. Never reduce target visibility, change geometry, or vary physics tick rate to meet a graphics budget. Qualify upgrades only if their visual benefit is perceptible at the driver camera distance.

Delivery

Bundle dependencies, fonts, robot assets, and WASM files locally. Host versioned static assets over HTTPS. Add an offline-capable service worker after initial online load; verify offline reload explicitly. Cache only application assets, never reports or driver records. A first-ever launch still requires the app to have been downloaded. Surface updates between sessions, pin a build for the whole session, and permit rollback to a previously qualified build.

If school filtering or offline installation rules prevent browser deployment, evaluate a packaged/native delivery route after confirming the exact restriction. Native delivery adds signing, macOS notarization, Windows installer behavior, and OS-specific testing; it does not automatically solve an unsupported controller.

17. Verification and validation plan

Engineering verification

Area	Evidence required before release
Math and scoring	Coordinate signs, wheel-speed desaturation, deadzone/ramp behavior, anchor boundaries, DNF/invalid handling, penalties, rounding, aggregate eligibility
Physics	Straight-line acceleration, speed cap, coast/brake, pivot, swerve steering lag, mecanum strafe/diagonals, traction saturation, impacts, no tunneling
Test runner	Ordered gates, reverse crossings, bumper containment, dwell reset, timeout, wrong-way measurement, cue presentation, invalidation
Devices	Plug/unplug, reconnect, unknown mapping, axis drift, keyboard ghosting, focus loss, browser resize, device change
Reports/session	Reload recovery, expired session, storage denial, cleared data, Letter/A4, grayscale, long text, batch pagination, HTML/PDF numeric agreement
Deployment	Fresh install/load, cached/offline reload, build update isolation, unsupported graphics, school-network access
Visual quality	Finished art on both platforms; material scale, shadows, clear targets, stable detail transitions, no placeholder UI; memory/performance within budget

Use unit tests for pure transforms and scores; headless physics fixtures for motion; scenario tests for start-to-report behavior; browser automation for menus/storage/export; physical hardware tests for controller compatibility and responsiveness. Synthetic gamepad tests cannot establish real USB support.

Physics acceptance proposals

Against repeatable measured real-robot runs, target steady speed within 10%, acceleration time within 15%, stopping distance within 15%, and turn/strafe response within 15% of the selected reference metrics. Use absolute tolerances near zero. These are initial training-fidelity goals, not certifications; adjust based on measurement uncertainty and coaching impact. Keep uncalibrated presets clearly labeled even if mathematical tests pass.

Verify changing render rates between 30, 60, and 120 Hz preserves same-input physics trajectories within agreed tolerance; an assessment at 30 fps can still be unqualified for user responsiveness. Test long stationary contact, high-speed gate crossing, saturation, and impossible starting poses explicitly.

Assessment pilot

Recruit approximately 12-20 students with varied driving experience plus two coaches if available. Standardize hardware and instructions. Run the full suite twice, counterbalance order, record prior experience, and compare score spread, ceiling/floor effects, and between-session changes. A starting review trigger is median skill-score change above 10 points without intervening training; investigate before claiming stable measurement.

Have coaches independently assess corresponding real-robot drills without first seeing simulator scores. Compare task-level outcomes and disagreements. This is an exploratory pilot; a small sample cannot establish population norms or broad predictive validity. Review if the app mostly measures familiarity with video games or field-relative assists. Revise anchors, publish a new rubric version, and do not silently mix old and new results.

Release gate: all functional requirements pass, both target platforms have a documented controller configuration, every drill is feasible, score calculations are reproducible, and unresolved limitations appear in the release notes/report context.

18. Implementation plan and effort

Milestones with exit criteria

Phase	Deliverable and dependency	Estimated developer-weeks
0 - Feasibility	Controller/graphics matrix; bare arena; motion sketches for all three drives; one PDF; select stack	1-2
1 - Driving foundation	Fixed loop, force models, collisions, calibrated controls, cameras, free drive, representative finished art scene; depends on 0	3-4
2 - Vertical assessment slice	T01 end to end: setup, trial, telemetry, score, session recovery, PDF; depends on 1	2-3
3 - Core battery	T02-T06, matching rules, full-suite aggregation, coach review; depends on 2	3-4
4 - Pilot and release	Hardware QA, physics tuning, usability/pilot changes, offline delivery, accessibility/report QA	3-5
Total	First release with three drivetrains and six core tests	12-18

The 12-18 developer-week estimate covers software engineering with an established visual direction and usable assets. Budget an additional 2-4 person-weeks for art/UI design, modeling, materials, lighting, and visual optimization: approximately 14-22 person-weeks total, or 490-770 hours at 35 productive hours per week. An artist can overlap engineering work; if one person performs both roles, use the combined range for scheduling.

These estimates assume regular FRC coach input, access to representative computers/controllers and a test robot, a small polished original asset set, and no seasonal mechanisms. Student part-time schedules and unfamiliar physics/3D work can extend calendar time substantially. Optional T07/T08 and richer replay tools follow the core release unless capacity allows them earlier. The visual-quality requirement is part of the first release, not postponed as optional polish.

First ten implementation tasks

1. Scaffold TypeScript/Vite, lint/type checks, test runner, and schema validation.
2. Build input inspector and keyboard/gamepad normalization; record hardware matrix.
3. Establish world coordinates, camera framing, fixed timestep, and performance panel.
4. Implement swerve contact-force prototype and free-drive controls.
5. Prototype differential and mecanum; confirm their distinct motion and feasibility.
6. Implement bumper contacts, gate geometry, and start/reset state handling.
7. Define T01 data, metric reducers, score tests, and result explanations.
8. Build ReportModel, local PDF/print output, and temporary session recovery.
9. Run an end-to-end coach/student trial and measure input-to-display response.
10. Freeze the initial interfaces and implement the remaining core tests.

Costs and maintenance

The architecture needs static hosting rather than a paid application backend. Hosting may be available through existing team/school resources; verify local policies and current provider limits before promising a cost. Hardware testing requires the actual USB controllers, compatible data cables/adapters, and access to Windows and Mac stations. Labor and calibration access are likely the largest costs.

Maintain dependency locks, asset/license attribution, automated score tests, a controller compatibility list, and a changelog separating physics changes from rubric changes. Requalify representative stations after substantial browser/OS or engine updates. Include a coach setup guide and student quick-start in the release.

19. Risks, decisions, and build-versus-use

Risk	Response / decision trigger
Looks realistic but trains the wrong motion	Calibrate measurable response and review with drivers before expanding tests
USB controller differs by OS/browser	Complete model-specific hardware spike first; qualify supported combinations
Slow computers distort assessment	Preflight benchmark, frozen settings, latency evidence, invalidation on stalls
Scores reward gaming the course	Ordered swept gates, complete-footprint checks, minimum entry speeds, explicit DNF
Cross-profile scores seem comparable	Compatibility key and visible report context; no combined ranking
Students lose reports on shared machines	Clear export workflow, reload recovery, batch PDF, explicit expiry/clear action
Scope expands into a seasonal game	Keep mechanisms/multiplayer outside the first release
Test scores are mistaken for driver selection proof	Pair with coach observations and real-robot drills

Why build this rather than only use an existing simulator?

Use an existing simulator if the main need is enjoyable seasonal driving and match familiarity. This project is justified when the main need is structured assessment: defined conditions, interpretable measurements, transparent scoring, and portable reports. Existing products can remain useful alongside it.

Before full implementation, conduct a short hands-on review of xRC and MoSim with the team's controller. Look specifically for fixed-camera drills, repeatable test configuration, raw metric export, and coach report workflows. This document's primary-source website review establishes product positioning, not a complete competitive feature audit. If a product already meets the assessment need, integration or a complementary coach workflow may be cheaper than a new simulator.

Questions to settle during the feasibility phase

- Which exact Windows/Mac computers and USB controller models must pass?
- What robot measurements can the team provide for calibration?
- Is the first pilot focused on novice instruction, tryouts, or both?
- Which aids mirror the team's normal drive configuration?
- Are school-network restrictions compatible with HTTPS static hosting and cached use?

These do not block the product/architecture recommendation. They determine the release's tested compatibility list, calibration quality, and final rubric.

Definition of done

A coach on either qualified platform can connect a documented USB controller, calibrate it, drive all three presets, run six core tests, inspect valid and incomplete outcomes, export a legible report, switch students, and end the session. Keyboard operation follows the same workflow in its own comparison group. The release records its physics limitations and has completed the verification and pilot review described here.

20. Sources and evidence boundaries

Primary product and technical documentation reviewed September 11, 2026. Links also appear beside the relevant claims. Product pages are descriptions by their publishers; recommendations, numerical defaults, rubrics, milestone estimates, and architecture choices are proposed design work.

Source	What it supports
MoSim	Season-based FRC simulator, real-team robots, scores, and modding positioning
xRC Simulator	Seasonal simulator releases
xRC server list	Multiplayer server offering
DSIM	Browser-based 2D FTC simulator positioning
Three.js WebGLRenderer	WebGL 2 rendering API
Three.js materials	Physically based metallic/roughness materials and environment lighting
Three.js glTF loader	glTF assets and supported compression/instancing extensions
Rapier JavaScript setup	WASM/JavaScript physics integration and separate rendering responsibility
Rapier determinism	Conditions and caveats for deterministic simulation
Godot web export	Web export requirements and input/platform considerations
Unity web input	Browser gamepad input and interaction requirements
MDN Gamepad controls	Browser controller discovery, mappings, polling, and events
Mozilla Gamepad security	HTTPS/secure-context and embedding policy considerations
Apple wired controller guidance	Compatible wired Mac controller connections
Apple Xbox guidance	Listed Xbox wireless models and Bluetooth connection procedure
WPILib kinematics	Differential, swerve, and mecanum reference families
WPILib swerve kinematics	Module velocity/angle conversion and field-relative behavior
pdf-lib	Browser-compatible local PDF creation
MDN sessionStorage	Tab-session storage and lifecycle behavior

Evidence not yet collected: controller compatibility results, real-robot measurements, measured application performance, hands-on competitor feature comparisons, and student assessment validity. Those are explicit feasibility/pilot deliverables, not findings of this document.